

Chapter 16 — Adversarial Red-Team Simulation Agent



Feature summary: An AI-driven “opponent” — the **Red-Team Agent (RTA)** — injects complications into a live run: blocking routes, escalating incidents, straining resources, and timing disruptions for maximum training pressure. It adapts its intensity to how well the trainee is performing, turning static scenario playback into **genuine training under pressure**.

16.1 Concept & Motivation

Scripted scenarios are predictable: run the same YAML twice and a trainee learns the script, not the skill. The RTA adds a **reactive adversary** that observes the world state and chooses *when and where* to make things harder — so every run is a fresh exercise and coordination under stress becomes trainable.

Crucially, this is **not** a military opponent model: the RTA plays the role that exercise designers call the “exercise controller” or “white cell” — injecting realistic complications (road blocked by debris, secondary incident, comms degradation, supply spoilage) exactly like human exercise controllers do in real disaster drills.

16.2 The Golden Rule Still Holds



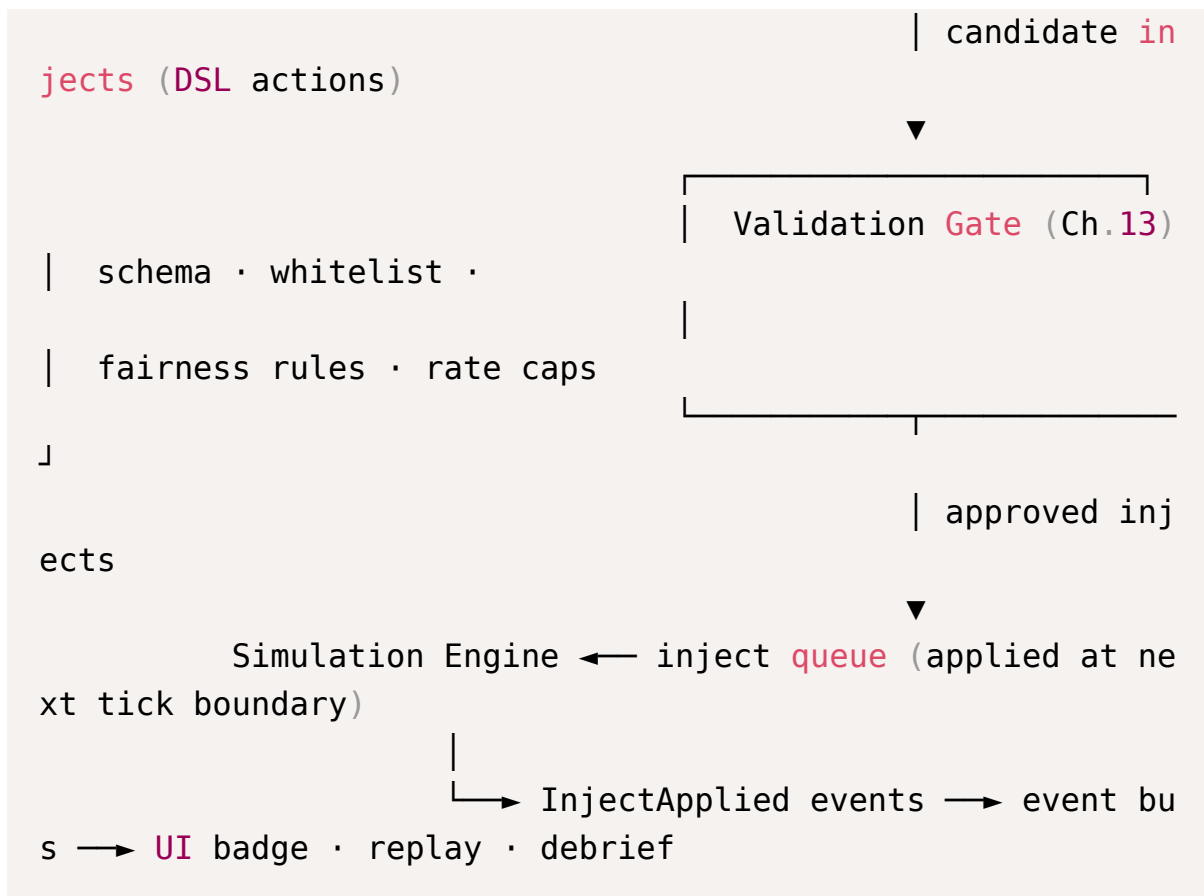
The RTA is a scenario author at runtime, never a physics engine. It emits only whitelisted scenario DSL actions (Ch. 7) through the same **Validation Gate** used by the AI scenario author (Ch. 13 §13.5). The engine remains the sole computer of outcomes (ADR-6). The RTA cannot teleport units, edit world state, or invent event types — it can

only *inject legal scenario actions*, exactly as a human scenario file could.

Determinism is preserved by design: every RTA decision is recorded as an `InjectApplied` event (with the decision context and RNG state), so **replaying a run replays the exact same injects at the exact same ticks** — the golden-run suite still works, and debriefs can show “why the adversary did that.”

16.3 Architecture





The three RTA components

1. **Assessor** — computes a rolling trainee performance score from live metrics (incident acknowledgment latency, response-time percentile vs target, resource discipline, task completion). Pure functions over the event stream; fully testable.
2. **Director policy** — decides target intensity on a 0–10 scale using a *flow-corridor* rule: if the trainee is coasting (score high, few open incidents), raise pressure; if overwhelmed (score collapsing, alert backlog), ease off. v1 is a transparent rule table; v2 optionally lets an LLM choose *narrative flavor* (which inject fits the story) while the rule table still owns *intensity* — keeping difficulty fair and explainable.
3. **Inject planner** — selects from the **Inject Playbook** (below), choosing target and timing (e.g., block the route *currently being used* by the responding unit — maximum training value, found via a PostGIS query on active mission paths).

16.4 The Inject Playbook (v1)

Inject	DSL effect	Trains
Route blockage	spawn zone (impassable, timed) on an active route	replanning, rerouting
Secondary incident	spawn incident near an in-progress response	prioritization, splitting forces
Incident escalation	raise severity of an unacknowledged incident	triage discipline
Comms degradation	delay event visibility for one sector (UI-side, timed)	operating with stale data
Resource spoilage	reduce a depot stock by %	logistics replanning
Weather worsening	intensify an existing weather cell	feasibility reassessment
Sensor ghost	emit a false detection (marked internally)	verification habits, healthy skepticism
Casualty surge	batch of minor-severity incidents in one sector	mass-triage workflow

Each playbook entry declares: preconditions, cooldown, max uses per run, intensity cost, and **fairness constraints** (see §16.5). Adding a new inject = adding a playbook entry + DSL actions; no engine changes.

16.5 Fairness Rules (enforced by the Validation Gate)

Training collapses if the adversary feels arbitrary. Hard limits, checked before any inject is applied:

- **No no-win states:** an inject may not block *all* feasible routes to an active critical incident, or drop any resource below the minimum needed for currently active missions.
- **Rate caps:** ≤ 1 major inject per 5 sim-minutes; ≤ 3 concurrent active injects; global intensity budget per run (configurable per difficulty tier).
- **Telegraphing:** every inject produces observable evidence *before or as* it lands (e.g., weather worsens over 60 s rather than instantly) — skilled trainees can anticipate.
- **Transparency after the fact:** all injects are visible in debrief with the RTA's reasoning ("score 8.2, pressure raised; chose route blockage on M-3's active path").

16.6 Functional Requirements

- FR-16.1: RTA can be enabled per run with difficulty tiers (`training` / `standard` / `intense`) mapping to intensity budgets and playbook subsets.
- FR-16.2: Assessor publishes a trainee performance score (updated $\leq$ every 30 sim-seconds) visible to instructors/observers only.
- FR-16.3: All injects pass the Validation Gate (schema, whitelist, fairness, rate caps); rejected injects are logged with reasons.
- FR-16.4: Every applied inject emits `InjectApplied` with decision context; replays reproduce injects tick-exactly.
- FR-16.5: Injects are visually distinct in the instructor view (badge) but **indistinguishable from organic events for trainees** during the run.
- FR-16.6: Debrief view: timeline of injects vs performance score, with the director's reasoning per inject (Ch. 15 debrief integration).
- FR-16.7: Kill switch: commander/instructor can pause or disable the RTA mid-run; disable is itself an event.
- FR-16.8: War-room integration: with multiple operators (Ch. 15), the RTA targets the *weakest-covered sector*, training load balancing across the team.
- FR-16.9: The sensor-ghost inject is always internally labeled `synthetic_false` ; scoring rewards trainees who flag it, and debrief reveals it.

16.7 Implementation Plan

Sprint	Work
16-A	Assessor metrics + score function (pure, unit-tested); <code>InjectApplied</code> event type; instructor score widget
16-B	Playbook framework (preconditions/cooldowns/fairness constraints as data), 4 core injects (blockage, secondary incident, escalation, weather)
16-C	Director rule policy + intensity budget + kill switch; Validation Gate fairness checks; determinism/replay tests
16-D	Remaining injects, debrief timeline UI, war-room weakest-sector targeting, difficulty tiers; optional LLM narrative selector (flagged experimental)

Effort: ~4 sprints (5–6 weeks). Depends on: engine + DSL (Ph. 1–2), Validation Gate (Ch. 13), metrics (Ph. 6 for full Assessor; a reduced score works from Ph. 2 data). Recommended slot: **after Chapter 13's What-If Theater** (they share the Validation Gate), i.e., Phase 6.5–8 territory; pairs best when built alongside War-Room Mode (Ch. 15) for team training.

16.8 Testing Additions

- **Determinism:** golden run with RTA enabled — same seed $\Rightarrow$ identical inject sequence and event-stream hash (1x vs 20x speed).
- **Fairness property tests:** generated world states $\Rightarrow$ no-win detector never passes an inject that isolates a critical incident or starves active missions.
- **Flow-corridor simulation test:** scripted “perfect trainee” and “frozen trainee” bots $\Rightarrow$ assert intensity rises/falls within bounds.
- **Replay fidelity:** debrief inject timeline matches live run exactly.

16.9 Risks

Risk	Impact	Mitigation
RTA feels unfair/random → trainees disengage	High	Fairness rules in the Gate (hard), telegraphing, full debrief transparency
Difficulty oscillation (pressure whiplash)	Medium	Hysteresis in the director policy; intensity changes rate-limited
LLM narrative selector breaks determinism	Medium	LLM chooses among pre-validated candidates only, choice recorded in <code>InjectApplied</code> ; feature-flagged off by default
Inject sprawl bloats the engine	Medium	Injects live in the playbook + DSL only; engine untouched (golden rule)
Score gaming (trainees learn the metric, not the skill)	Low	Multi-factor score, periodic weight review, instructor override